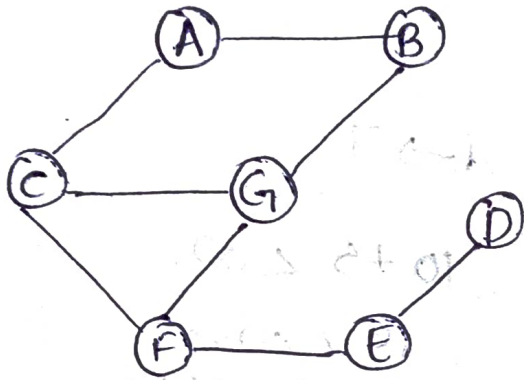


BFS:

→ BFS visits a graph level by level. First start vertex, then all of its neighbours, then their neighbours and so on. It uses a queue and a visited set.

1.



Adjacency List:

A[B, C]

B[A, G]

C[A, G, F]

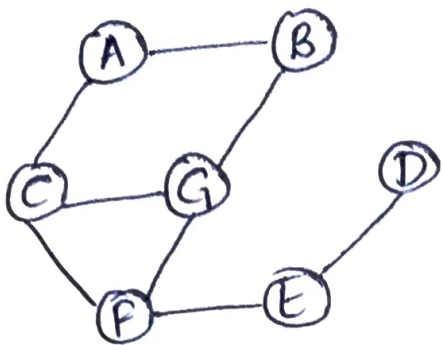
D[E]

E[D, F]

G[B, C, F]

F[G, C, E]

Step 1:



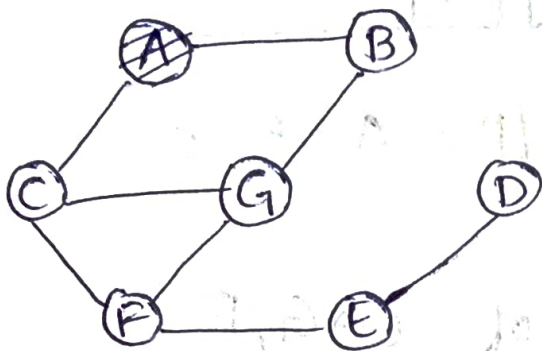
Queue



Enqueue A

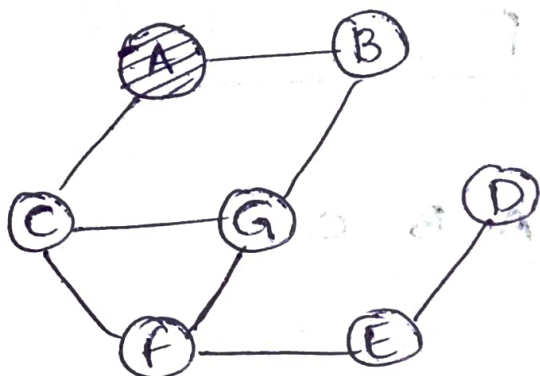
Step 2:

Dequeue front.

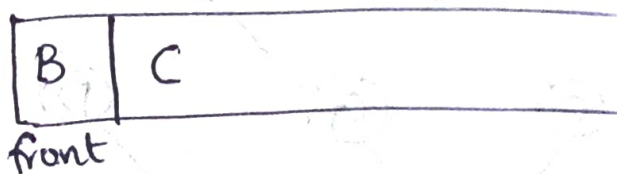


O/p :- A

Step 3: Unvisited Neighbours of A $\rightarrow$ B, C

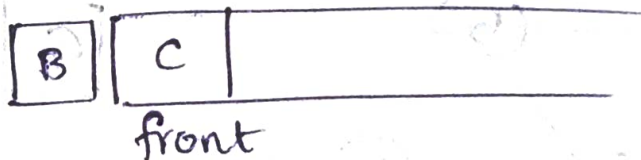
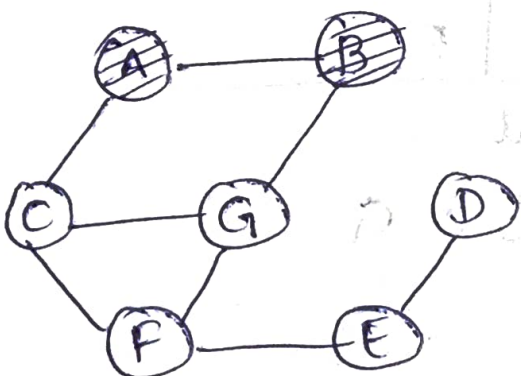


Enqueue B, C



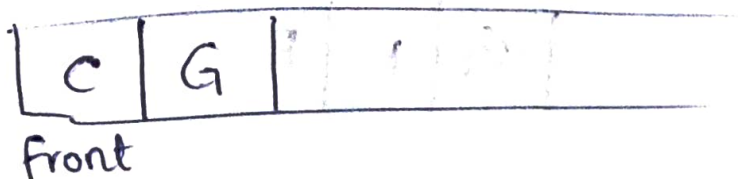
O/p :- A

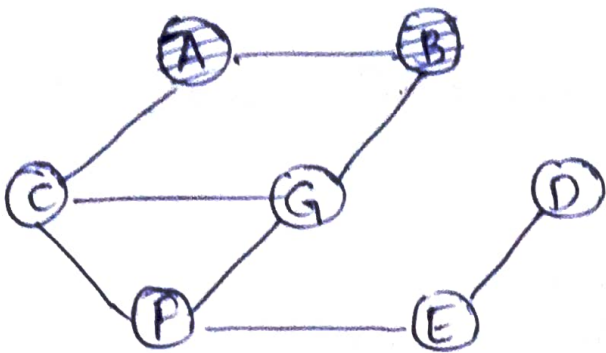
Step 4: Dequeue front



O/p :- A B

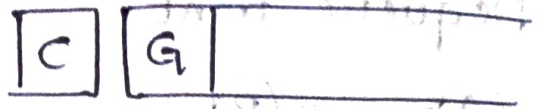
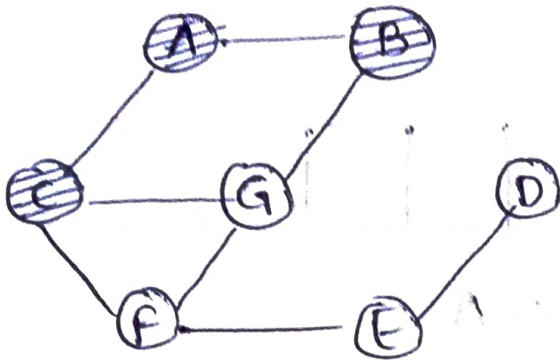
Step 5: Unvisited Neighbours of B $\rightarrow$ G Enqueue G





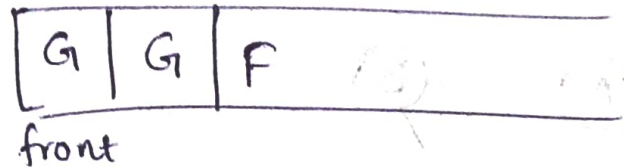
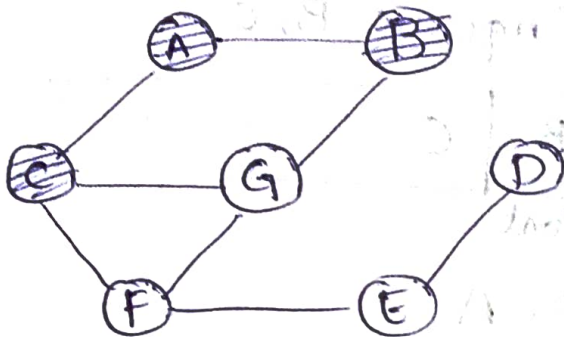
O/p:- A B

Step - 6: Dequeue front



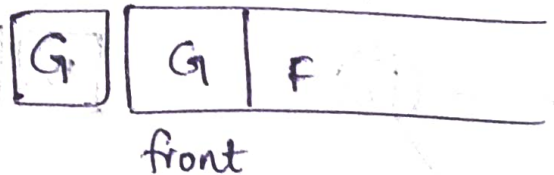
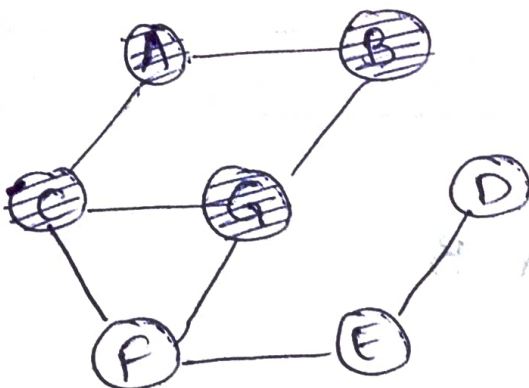
O/p:- A B C

Step 7: Unvisited Neighbours of $C \rightarrow G, F$
Enqueue G, F



O/p:- A B C

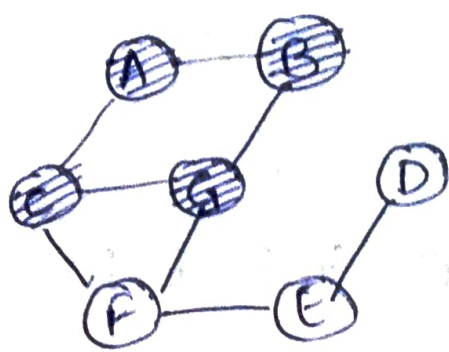
Step 8: Dequeue front



O/p:- A B C G

Step 9: Unvisited Neighbours of $G \rightarrow F$
Enqueue F

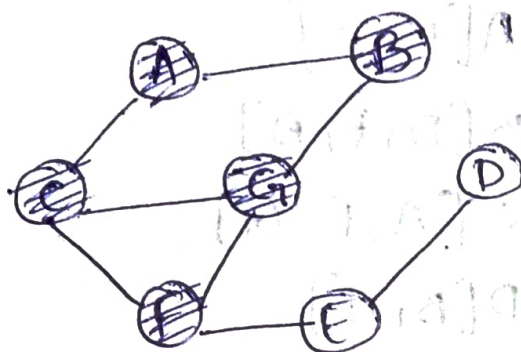




Step 10: Dequeue front



Step 11: Dequeue front



O/P:- A B C G F

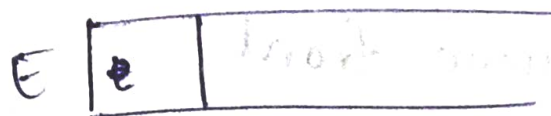
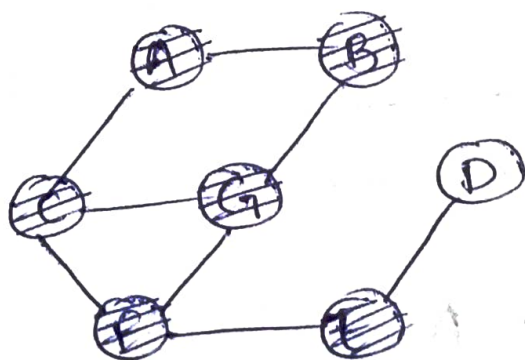
Step 12: Unvisited neighbours of F → E
Enqueue E



Step 13: Dequeue front



Step 14: Dequeue front



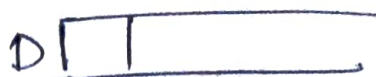
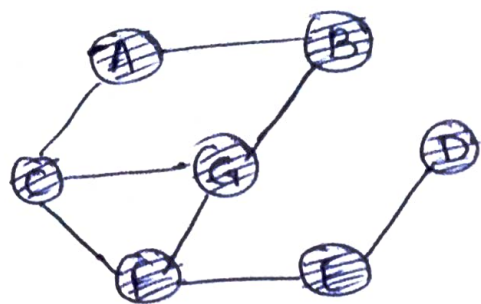
O/P:- A B C G F E

Step 15: Dequeue front

Unvisited neighbours of E → D
Enqueue D



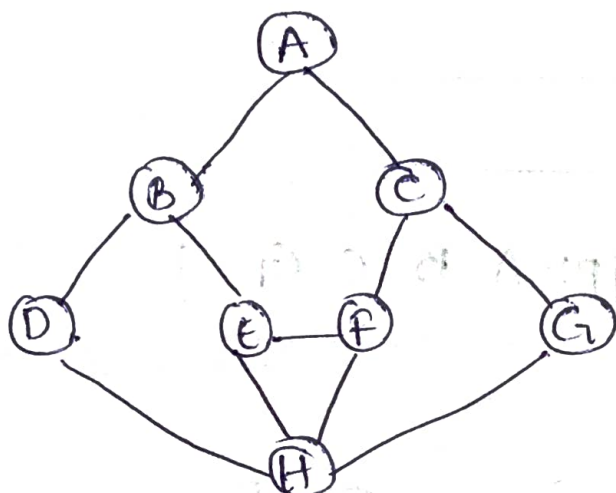
Step 16: Dequeue front



O/P:- A B C G F E D

BFS:

②



Adjacency list

A[B, C]

B[D, E, A]

C[A, F, G]

D[B, H]

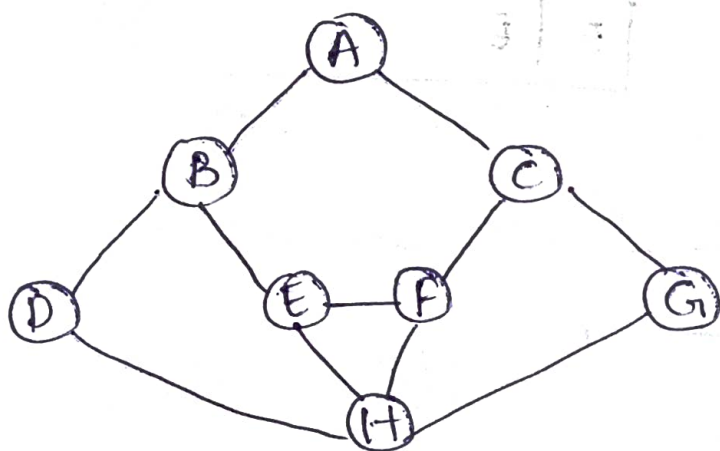
E[B, F, H]

F[C, E, H]

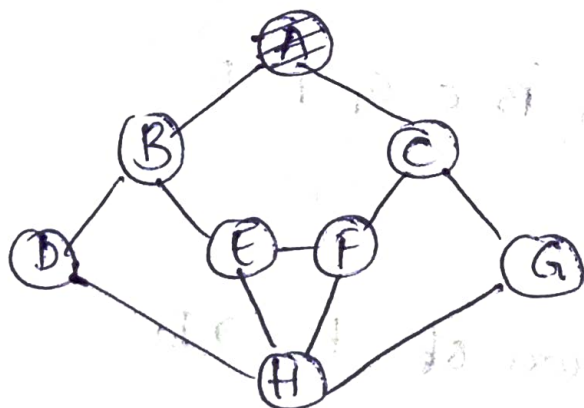
G[C, H]

H[E, F, D, G]

Step 1: Enqueue A

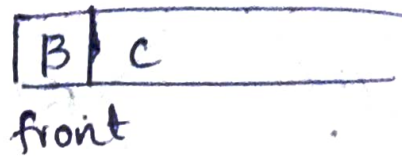


Step 2: Dequeue front

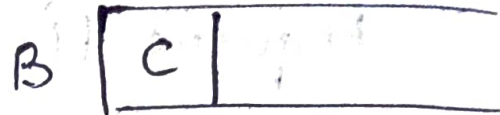
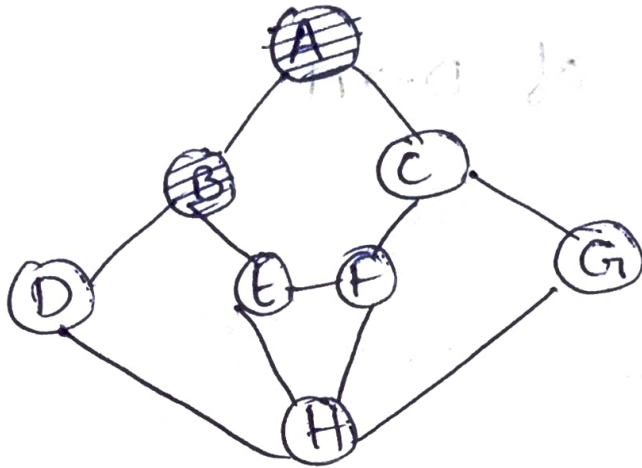


O/P:- A

Step 3: Unvisited Neighbours of A $\rightarrow$ B, C
Enqueue B, C

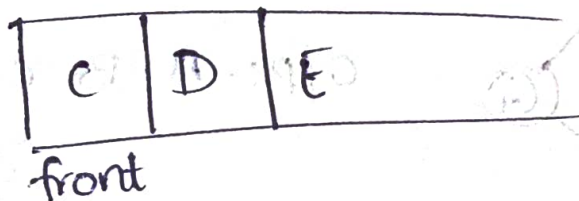


Step 4: Dequeue front

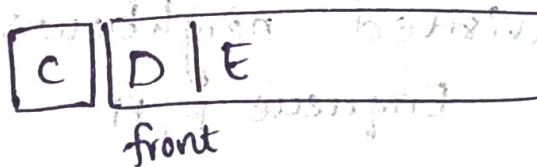
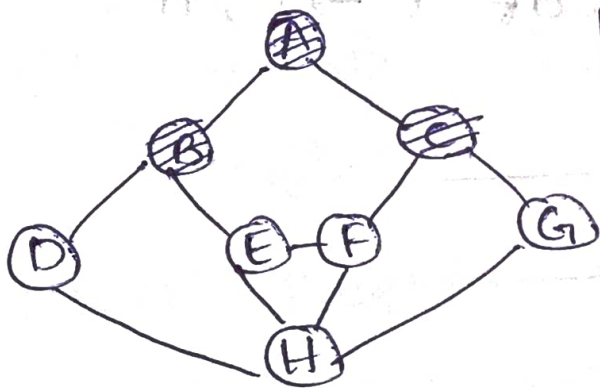


O/p:- A B

Step 5: Unvisited Neighbours of B $\rightarrow$ D, E
Enqueue D, E

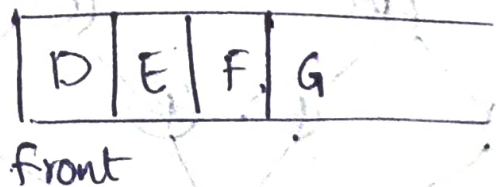


Step 6: Dequeue front

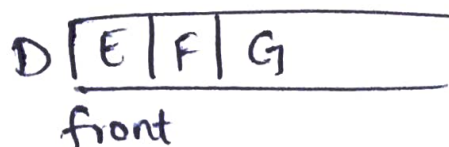


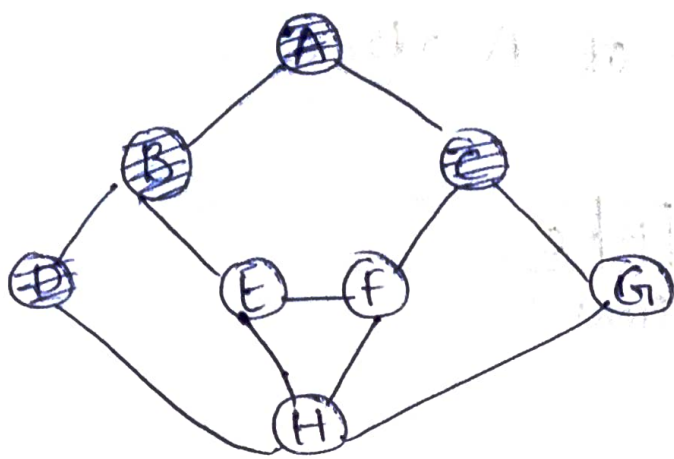
O/p:- A B C

Step 7: Unvisited Neighbours of C $\rightarrow$ F, G
Enqueue F, G



Step 8: Dequeue front



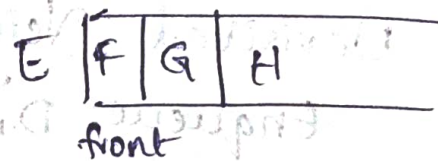
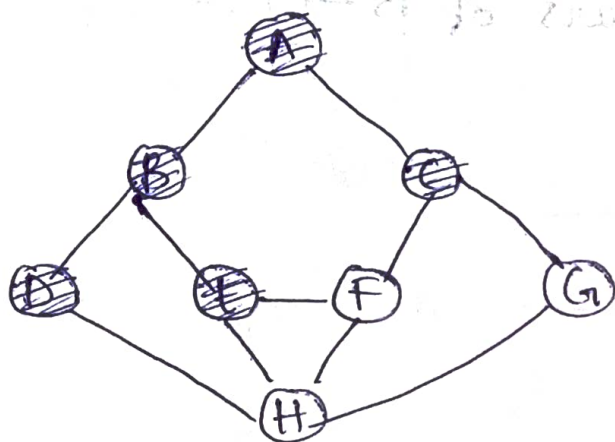


O/P:- A B C D

Step 9: Unvisited Neighbours of D $\rightarrow$ H
Enqueue H

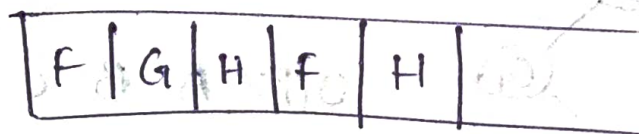


Step 10: Dequeue front

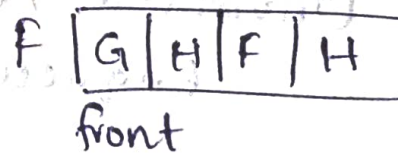
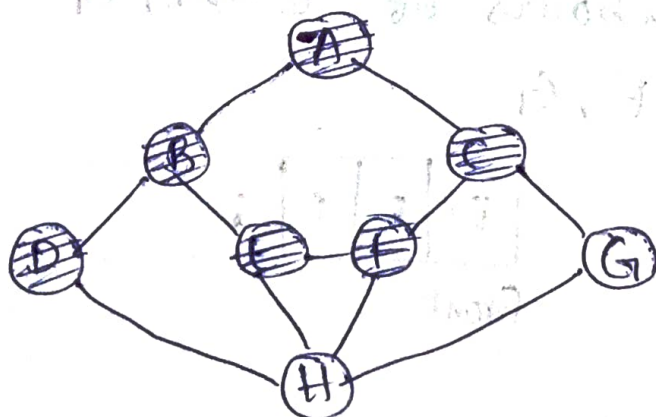


O/P:- A B C D E

Step 11: Unvisited neighbours of E $\rightarrow$ F, H
Enqueue F, H

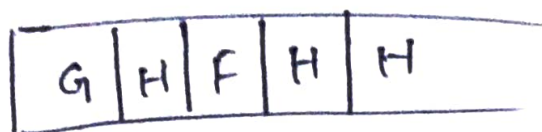


Step 12: Dequeue front

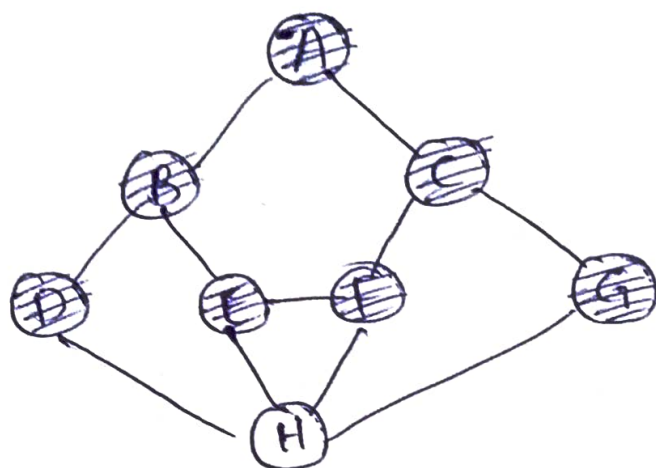


O/P:- A B C D E F

Step 13: Unvisited Neighbours of $F \rightarrow H$
 Enqueue H



Step 14: Dequeue front



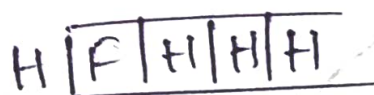
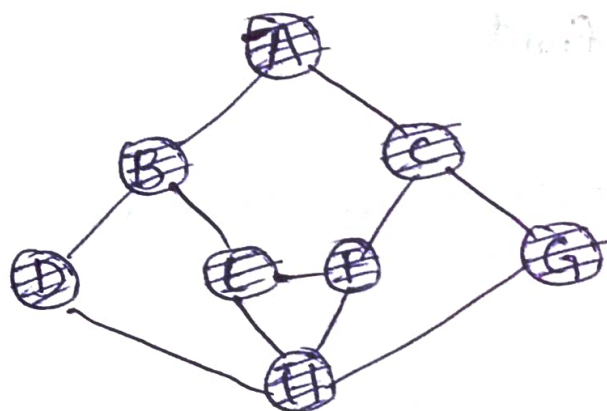
O/P:-

A B C D E F G

Step 15: Unvisited neighbours of $G \rightarrow H$



Step 16: Dequeue front



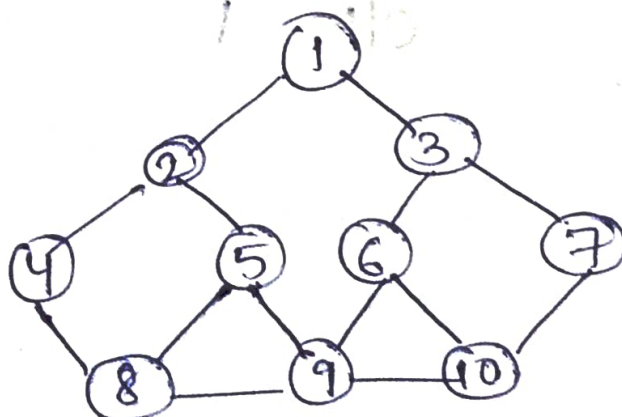
O/P:-

A B C D E F G H

Step 17: Dequeue front



③



Adjacency list:

1[2,3]

2[1,4,5]

3[1,6,7]

4[2,8]

5[2,8,9]

6[3,9,10]

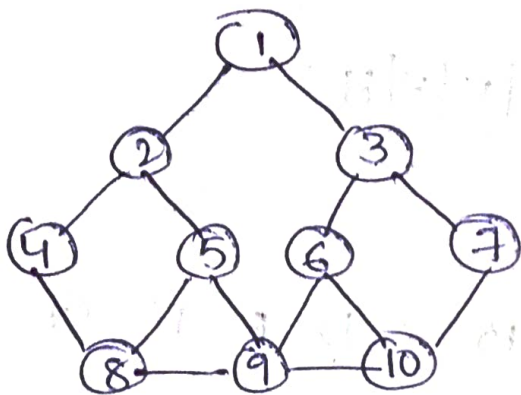
7[3,10]

8[4,5,9]

9[5,6,8,10]

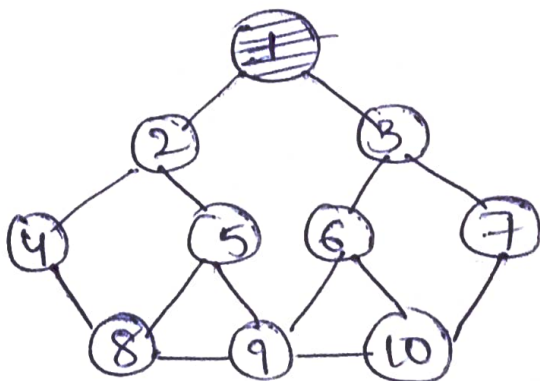
10[6,7,9]

Step 1: Enqueue 1



1
front

Step 2: Dequeue front



1

O/P:- 1

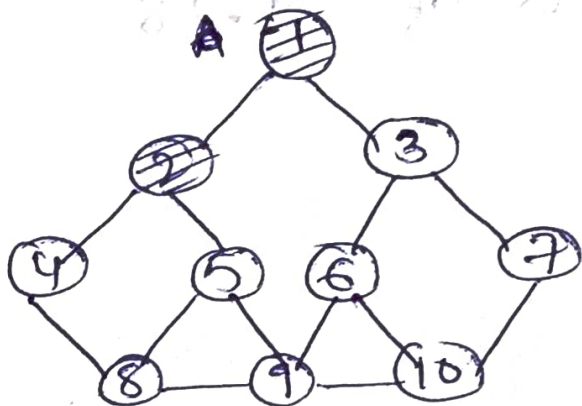
Step 3: Unvisited Neighbours of 1 $\rightarrow$ 2, 3

Enqueue 2, 3



front

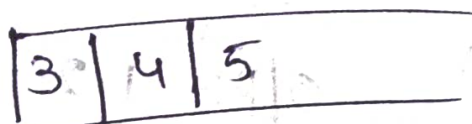
Step 4: Dequeue front



O/p: - 1, 2

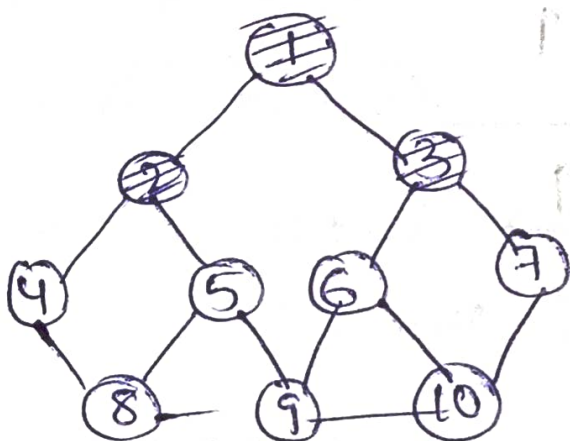
Step 5: Unvisited Neighbours of 2 $\rightarrow$ 4, 5

Enqueue 4, 5



front

Step 6: Dequeue front

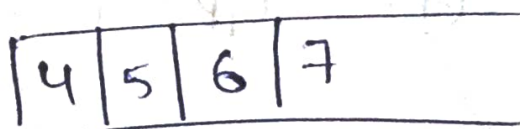


front

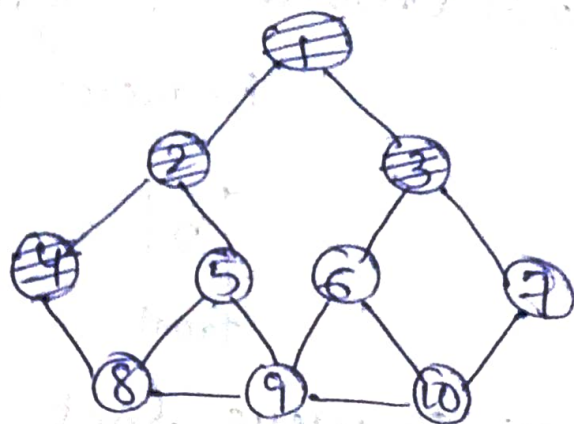
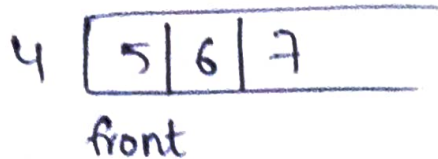
O/p: - 1, 2, 3

Step 7: Unvisited Neighbours of 3 $\rightarrow$ 6, 7

Enqueue 6, 7

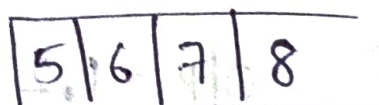


Step 8: Dequeue front

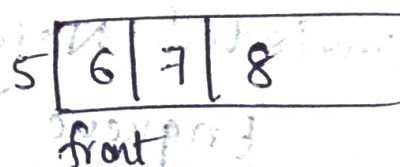
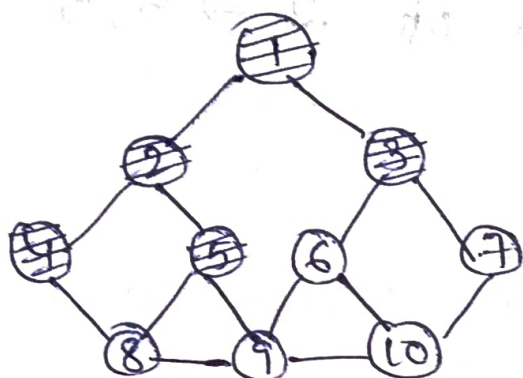


~~Step 8:~~ O/p:- 1 2 3 4

Step 9: Unvisited Neighbours of 4 → 8
Enqueue 8

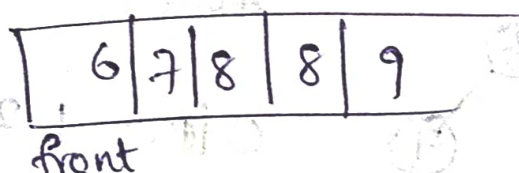


Step 10: Dequeue front

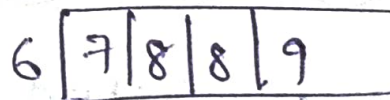
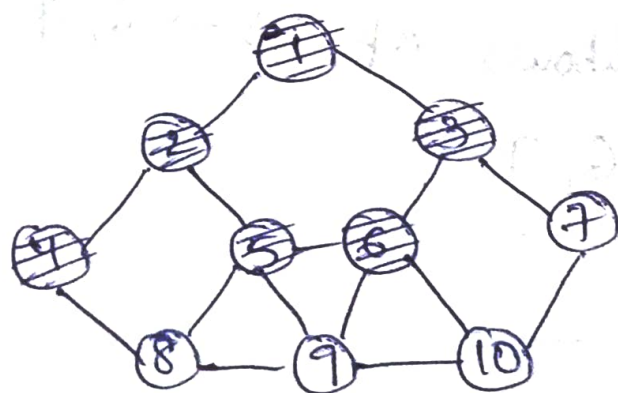


O/p:- 1 2 3 4 5

Step 11: Unvisited neighbours of 5 → 8, 9
Enqueue 8, 9



Step 12: Dequeue front

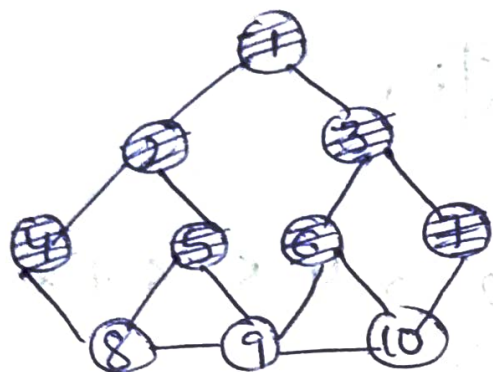


O/p:- 1 2 3 4 5 6

Step 13: Unvisited neighbours of 6 $\rightarrow$ 9, 10
Enqueue 9, 10

7	8	8	9	9	10
---	---	---	---	---	----

Step 14: Dequeue front



7

8	8	9	9	10
---	---	---	---	----

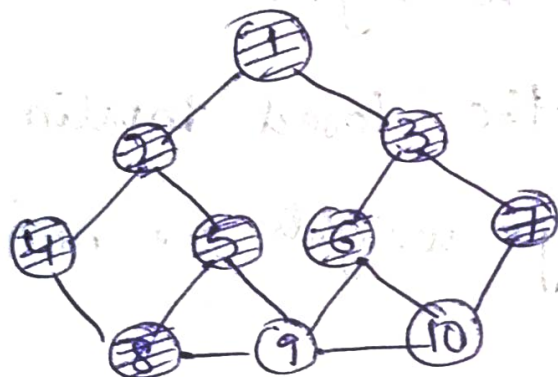
front

O/p:- 1 2 3 4 5 6 7

Step 15: Unvisited neighbours of 7 $\rightarrow$ 10
Enqueue 10

8	8	9	9	10	10
---	---	---	---	----	----

Step 16: Dequeue front



8

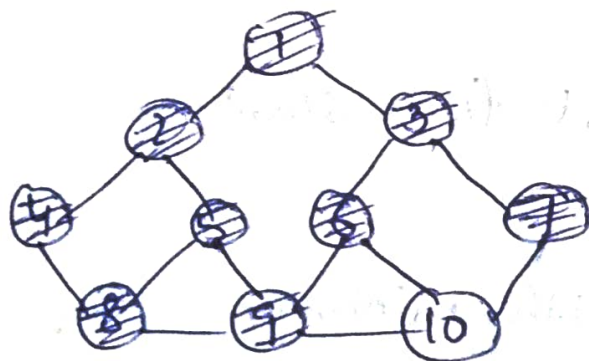
8	9	9	10	10
--------------	---	---	----	----

O/p:- 1 2 3 4 5 6 7 8

Step 17: Unvisited neighbours of 8 $\rightarrow$ 9
Enqueue 9

9	9	10	10	9
---	---	----	----	---

Step 18: Dequeue front



9

9	10	10	9
--------------	----	----	---

O/p:-

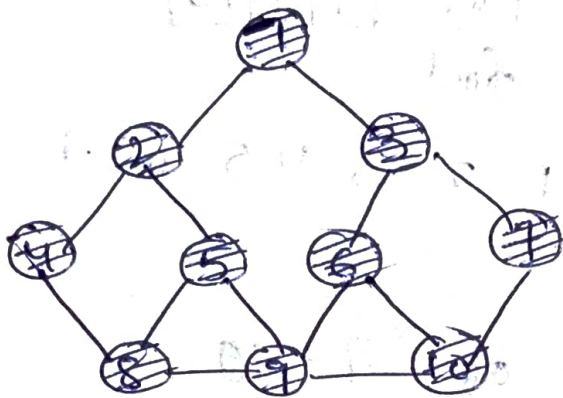
1 2 3 4 5 6 7 8 9

Step 19: Unvisited neighbours of 9 → 10

Enqueue 10

10	10	9	10
----	----	---	----

Step 20: Dequeue 10



10	10	9	10
----	---------------	--------------	---------------

O/P:- 1 2 3 4 5 6 7 8 9 10

Dijkstra's Algorithm:

- Finding the cheapest route from one node to every other node.
- 1956: Invented by Edsger W. Dijkstra.
- Greedy: Always suspends the closed invalid node.
- ≥ 0 : works when all edge weights are non-negative.

Problem:-

Given a weighted graph and a starting node, find the shortest path from the start to every other node. This is the single source shortest path problem.

- For every node: Shortest cost from start

1. cost table

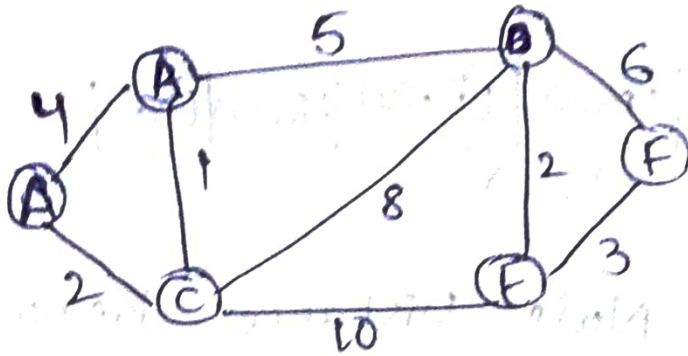
2. priority Queue: Node with minimum cost is popped.

3. Relaxation: For an edge, $u \rightarrow v$ if $\text{cost}[u] + w(u, v) < \text{cost}[v]$

Update - V's cost and previous node.

4. Decrease key

Q) Shortest path from A



if $d(u) + c(u, v) < d(v)$
then update

at A: $d(A) = 0$

$A \rightarrow B:$

$$0 + 4 < \infty \quad d(B) = 4$$

$A \rightarrow C:$

$$0 + 2 < \infty \quad d(C) = 2$$

$A \rightarrow D:$

$$0 + 8 < \infty \quad d(D) = 8$$

Node Distance Previous

A 0 -

B ~~∞~~ 4 A

C ~~∞~~ 2 A

D ~~∞~~ 8 -

E ∞ -

F ∞ -

(B, 4)	(C, 2)	
--------	--------	--

at C: $d(C) = 2$

$C \rightarrow A$

$$2 + 2 < 0 \quad (\times)$$

$C \rightarrow B$

$$2 + 1 < 4 \quad 3 < 4$$

$$d(B) = 3$$

$C \rightarrow D$

$$2 + 8 < \infty \quad d(D) = 10$$

$C \rightarrow E$

$$2 + 10 < \infty \quad d(E) = 12$$

Node Distance Previous

A 0 -

B ~~4~~ 3 A C

C 2 A

D ~~∞~~ 10 C

E ~~∞~~ 12 C

F ∞ -

(B, 3)	(D, 10)	(E, 12)
--------	---------	---------

At B: $d(B)=3$

$B \rightarrow A$

$$3 + 4 < 0 \quad (\times)$$

$B \rightarrow D$

$$3 + 5 < 10$$

$$8 < 10 \quad d(D)=8$$

$B \rightarrow C$

$$3 + 1 < 2 \quad (\times)$$

Node	Distance	Previous
A	0	-
B	3	C
C	2	A
D	8	B
E	12	C
F	∞	-

~~(B,3)~~ ~~(D,8)~~ (E,12) (D,8)

(B,3)

At D: $d(D)=8$

$D \rightarrow B$

$$8 + 5 < 3 \quad (\times)$$

$D \rightarrow C$

$$8 + 8 < 2 \quad (\times)$$

$D \rightarrow E$

$$8 + 2 < 12 \quad d(E)=10$$

$D \rightarrow F$

$$8 + 6 < \infty$$

$$14 < \infty$$

$$d(F)=14$$

Node Distance Previous

A	0	-
B	3	C
C	2	A
D	8	B
E	10	D
F	14	D

~~(E,12)~~ ~~(D,8)~~ (E,10) (F,14)

(D,8)

At E: $d(E)=10$

$E \rightarrow C$

$$10 + 10 < 2 \quad (\times)$$

$E \rightarrow D$

$$10 + 2 < 8 \quad (\times)$$

$E \rightarrow F$

$$10 + 3 < 14$$

$$d(F)=13$$

Node Distance Previous

A	0	-
B	3	C
C	2	A
D	8	B
E	10	D
F	13	E

~~(E,12)~~ ~~(F,14)~~ (F,13)

(E,10)

At F: $d(F) = 13$

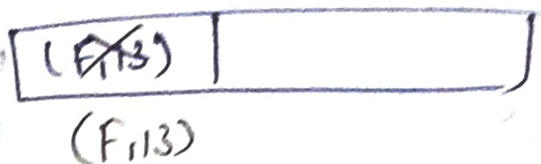
$F \rightarrow D$:

$$13 + 6 < 8 \quad (\times)$$

$F \rightarrow E$:

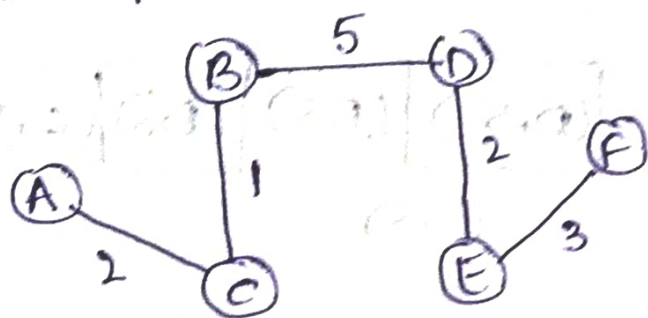
$$13 + 3 < 10 \quad (\times)$$

Node	Distance	Previous
A	0	
B	3	C
C	2	A
D	8	B
E	10	D
F	13	E

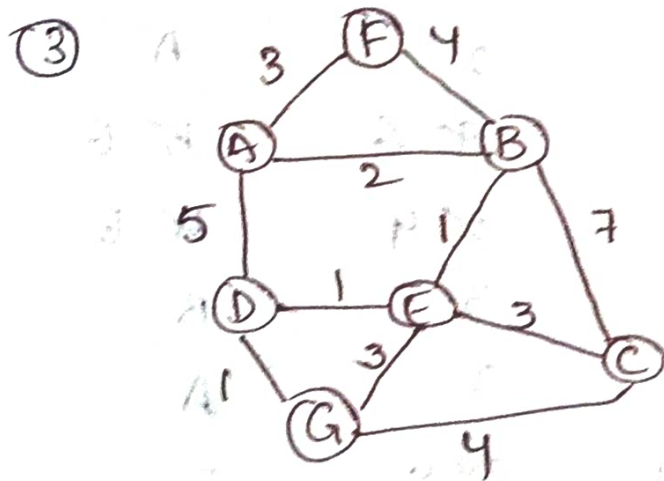


Shortest Distance

$A \rightarrow F$



$$2 + 5 + 1 + 2 + 3 = 13$$



$d(u) + c(u, v) < d(v)$ update

at A: $d(A) = 0$

$A \rightarrow B$:

$$0 + 2 < \infty \quad d(B) = 2$$

$A \rightarrow F$

$$0 + 3 < \infty \quad d(F) = 3$$

$A \rightarrow D$

$$0 + 5 < \infty \quad d(D) = 5$$

Node	Distance	Previous
A	0	
B	2	A
C	∞	
D	5	A
E	∞	
F	3	A
G	∞	

~~(B, 2)~~ | (D, 5) | (F, 3)

(B, 2)

at B: $d(B) = 2$

B $\rightarrow$ F:

$$2 + 4 < 3 \quad (\times)$$

B $\rightarrow$ A:

$$2 + 2 < 0 \quad (\times)$$

B $\rightarrow$ E

$$2 + 1 < \infty$$

$$3 < \infty$$

B $\rightarrow$ C

$$2 + 7 < \infty$$

$$9 < \infty$$

At E: $d(E) = 3$

E $\rightarrow$ B

$$3 + 1 < 2 \quad (\times)$$

E $\rightarrow$ D:

$$3 + 1 < 5$$

$$4 < 5 \quad d(D) = 4$$

E $\rightarrow$ G:

$$3 + 3 < \infty \quad d(G) = 6$$

E $\rightarrow$ C:

$$3 + 3 < 9$$

$$d(C) = 6$$

Node Distance Previous

A	0	
B	2	A
C	9	B
D	5	A
E	3	B
F	3	A
G	∞	

~~(D, 5)~~ | (F, 3) | ~~(E, 3)~~ | (C, 9)

(E, 3)

Node Distance Previous

A	0	
B	2	A
C	9	B
D	5	E
E	3	B
F	3	A
G	6	E

(D, 4) | (G, 6) | (C, 6) | ~~(E, 3)~~

(F, 3)

at F: $d(F) = 3$

F → A:

$$3 + 3 < 0 \quad (\times)$$

F → B:

$$3 + 4 < 2 \quad (\times)$$

Node	Distance	Prev
A	0	
B	2	A
C	6	E
D	4	E
E	3	B
F	3	A
G	6	E

~~(D, 4)~~ (G, 6) (C, 6)

(D, 4)

at D: $d(D) = 4$

D → A:

$$4 + 5 < 0 \quad (\times)$$

D → E:

$$4 + 1 < 3 \quad (\times)$$

D → G:

$$4 + 1 < 6$$

$$d(G) = 5$$

N	D	P
A	0	
B	2	A
C	6	E
D	4	E
E	3	B
F	3	A
G	5	D

~~(G, 5)~~ (C, 6)

(G, 5)

at G: $d(G) = 5$

G → D:

$$5 + 1 < 4 \quad (\times)$$

G → E:

$$5 + 3 < 3 \quad (\times)$$

G → C:

$$5 + 4 < 6 \quad (\times)$$

Node	Dist	Prev
A	0	
B	2	A
C	6	E
D	4	E
E	3	B
F	3	A
G	5	D

~~(G, 5)~~ (C, 6)

(C, 6)

At c: $d(c) = 6$

$C \rightarrow B:$

$$6 + 1 < 2 \quad (\times)$$

$C \rightarrow E:$

$$6 + 3 < 3 \quad (\times)$$

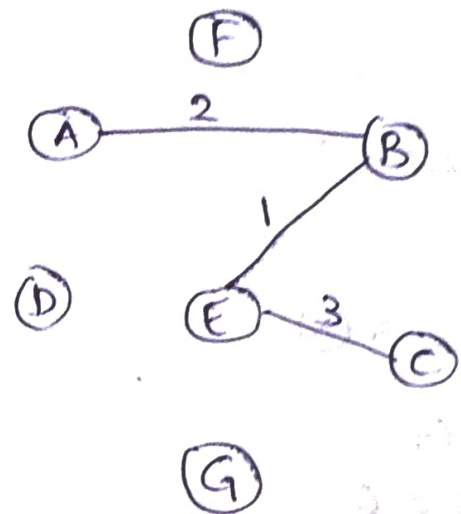
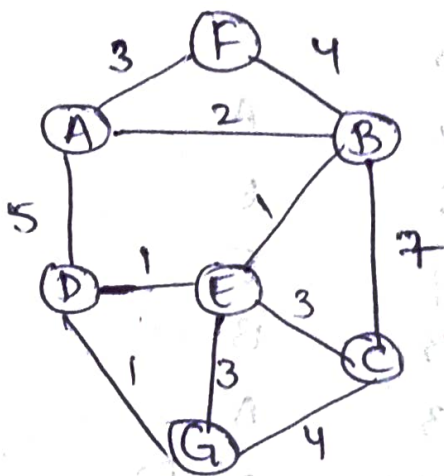
$C \rightarrow G:$

$$6 + 5 < 5 \quad (\times)$$

N	D	P
A	0	
B	2	A
C	6	E
D	4	E
E	3	B
F	3	A
G	5	D

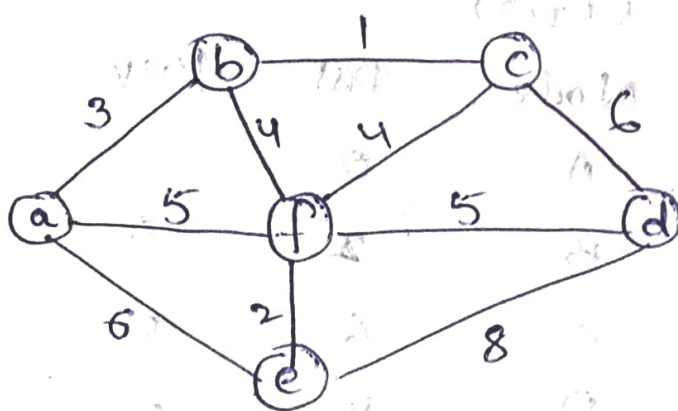
Shortest Distance:

from $A \rightarrow c$



$$2 + 1 + 3 = 6$$

Kruskal Algorithm:

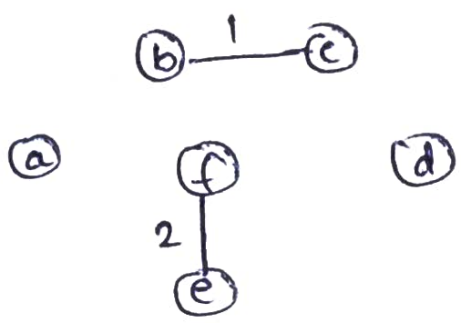


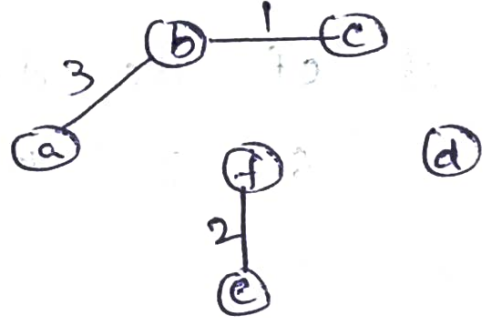
bc	ef	ab	bf	cf	af	df	ae	cd	de
1	2	3	4	4	5	5	6	6	8
✓	✓	✓	✓	×	×	✓	×	×	×

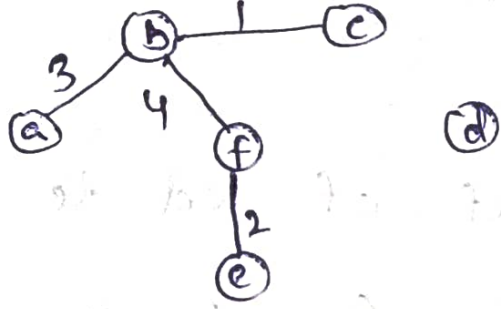
$|V| = 6$, $|E| = 10$, 5 edges

i,  no cycle ✓

 (bc)

ii,  no cycle ✓
(ef)

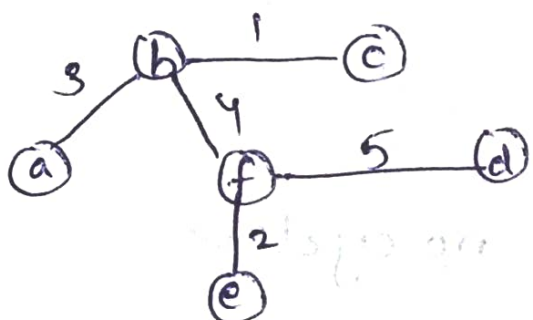
iii,  no cycle ✓
(ab)

iv,  no cycle ✓
(bf)

v, cf forms a cycle so skip

vi, af forms a cycle so skip

vii, df no cycle ✓



viii, ae forms a cycle so skip

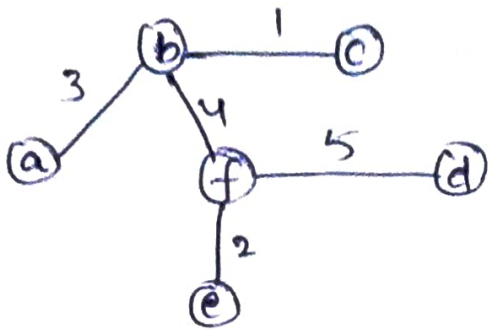
ix, cd forms a cycle so skip

x, de forms a cycle so skip

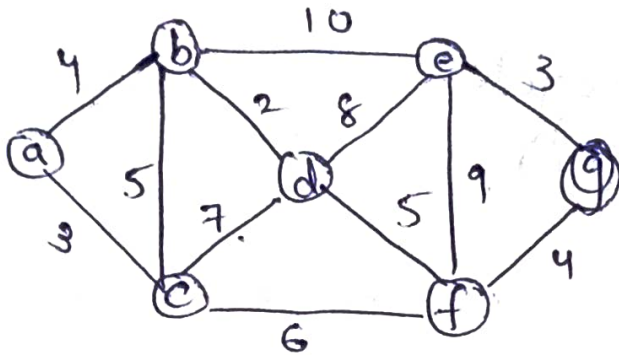
Minimum Spanning tree weight

$$= 1 + 2 + 3 + 4 + 5$$

$$= 15$$



2.

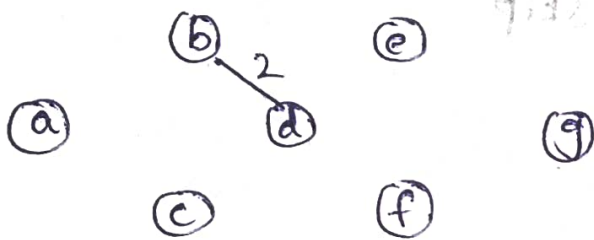


ab	ac	bc	bd	be	cd	cf	de	df
4	3	5	(2)	10	7	6	8	5
ef	eg	fg						
9	3	4						

Sorted:

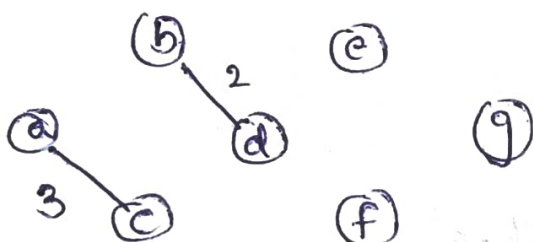
bd	ac	eg	ab	fg	bc	df	cf	cd	de	ef	be
2	3	3	4	4	5	5	6	7	8	9	10
✓	✓	✓	✓	✓	x	✓	x	x	x	x	x

i,



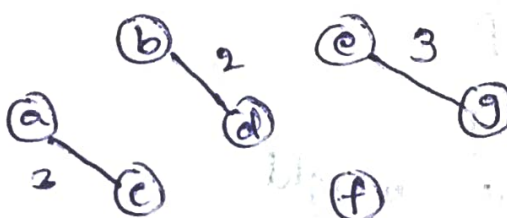
no cycle ✓

ii,

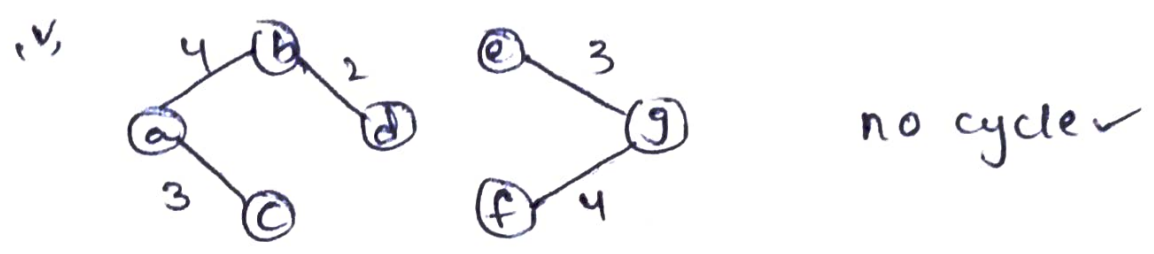
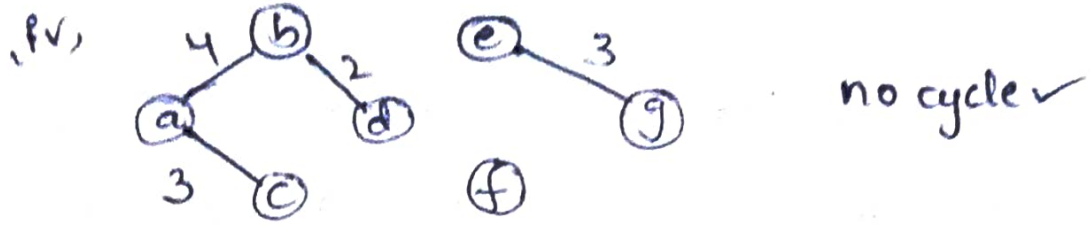


no cycle ✓

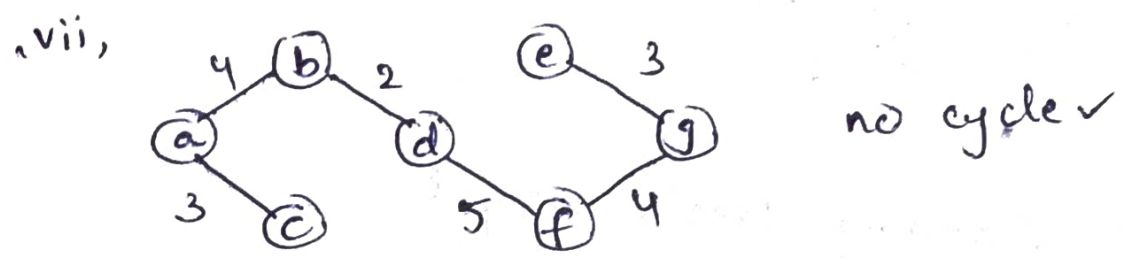
iii,



no cycle ✓



vi, bc forms a cycle so skip

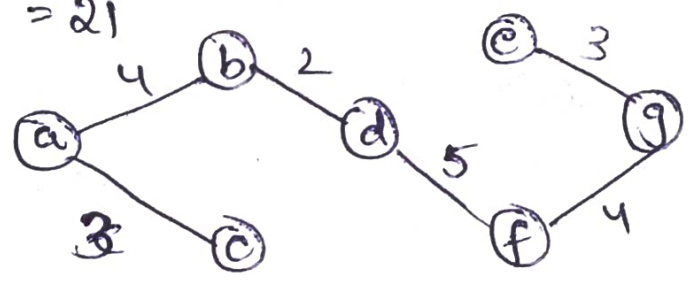


viii, cf, cd, de, ef, be form a cycle, so skip

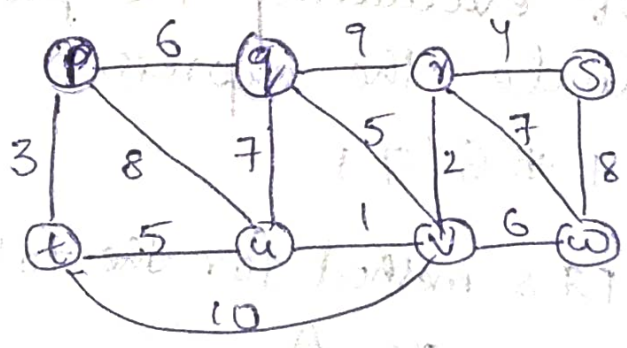
Minimum spanning tree's weight

$$= 2 + 3 + 3 + 4 + 4 + 5$$

$$= 21$$



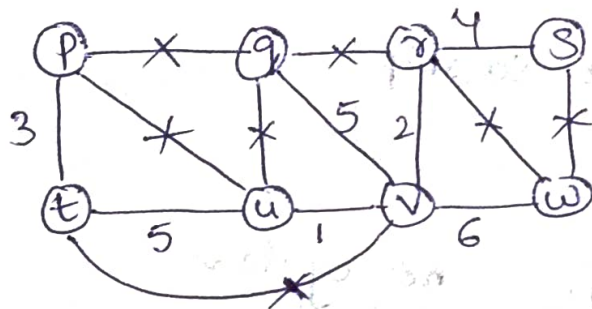
3.



p t	p u	p q	q r	q u	q v	r s	r w	s w	t u
3	8	6	9	7	5	4	7	8	5
x	x	x		x	x	x	x	x	x
t v	u v	v w							
10	1	6							
	x	x							

Sorted:

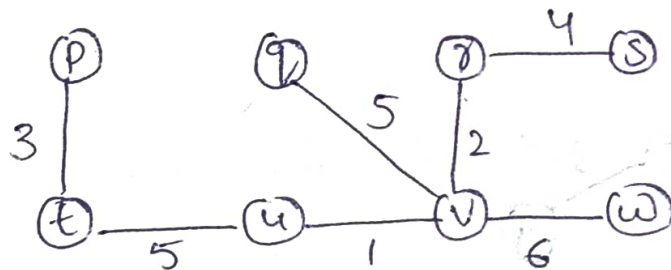
uv	rv	pt	rs	qv	tu	Pq	vw	qy	rw	pu	sw
1	2	3	4	5	5	6	6	7	7	8	8
✓	✓	✓	✓	✓	✓	x	✓	x	x	x	x
qr	tv										
9	10										
x	x										



Minimum spanning tree's weight

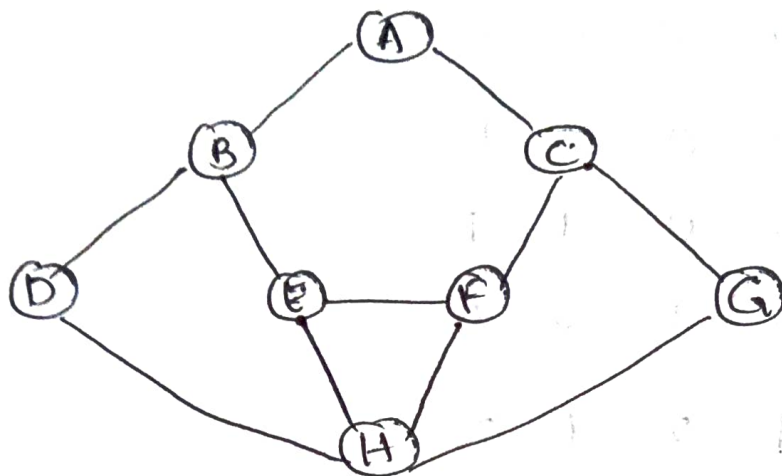
$$= 1 + 2 + 3 + 4 + 5 + 5 + 6$$

$$= 26$$



MST

2.



Adjacency list:

A: [B, C]

B: [A, D, E]

C: [A, F, G]

D: [B, H]

E: [B, F, H]

F: [C, E, H]

G: [C, H]

H: [D, E, F, G]

DFS order:

A, B, D, H, E, F, C, G

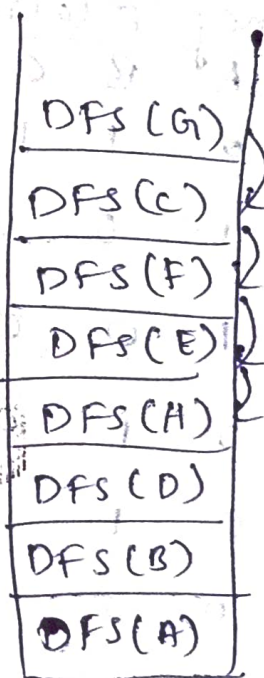
Steps:

1. All vertices are unvisited and the callstack is empty. DFS starts from vertex A.

2. Call DFS(A)

3. DFS(A) is pushed onto the call stack.

4. Visit A: mark A it is visited and add it to the DFS order.



5. Check neighbour B of A: not visited, so call DFS(B)
6. DFS(B) is pushed onto callstack.
7. Visit B: mark it is visited and add it to the DFS order.

8. Check neighbour A of B: already visited, so skip it.

9. Check neighbour D of B: not visited, so call DFS(D).

10. DFS(D) is pushed on to the callstack.

11. Visit D: mark it is visited and add it to the DFS order.

12. Check neighbour B of D: already visited, so skip it.

13. Check neighbour H of D: not visited, so call DFS(H).

14. Visit H: Mark it is visited and add it to the DFS order.

15. Check neighbour D of H: already visited, so skip it.

16. Check neighbour E of H: not visited. So call DFS(E).

17. Visit E: Mark it is visited and add it to the DFS order.

18. Check neighbour B of E: already visited, so skip it.

19. Check neighbour F of E: not visited, so call DFS(F).

20. Visit F: Mark it is visited and add it to the DFS order.

21. Check neighbour C of F: not visited, so call DFS(C).

22. Visit C: Mark it is visited and add it to the DFS order.

23. Check neighbour A of C: already visited, so skip it.

24. Check neighbour F of C: already visited, so skip it.

25. Check neighbour G of C: not visited, so call DFS(G)
26. visit G. Mark it is visited and add it to the DFS order.
27. Check neighbours C & H of G: already visited, skip it & pop(G).
28. Back to DFS(C): all neighbours are visited, skip it & pop(C).
29. Back to DFS(F): all neighbours are visited, skip it & pop(F).
30. Back to DFS(E): all neighbours are visited, skip it & pop(E).
31. Back to DFS(H): all neighbours are visited, skip it & pop(H).
32. Back to DFS(D): all neighbours are visited, skip it & pop(D).
33. Back to DFS(B): all neighbours are visited, skip it & pop(B).
34. Back to DFS(A): all neighbours are visited, skip it and DFS(A) returns and is popped.